

Atividade prática 2 - RMI

Reinaldo Kaminski Neto

I. INTRODUÇÃO E OBJETIVOS

Este trabalho tem como objetivo explorar o conceito de *Remote Method Invocation (RMI)* por meio da evolução do sistema *cat_babysitter*, originalmente desenvolvido como um *CRUD* no trabalho anterior. Nesta etapa, foi utilizada a biblioteca *Pyro5*, da linguagem *Python*, para permitir a invocação remota de métodos entre cliente e servidor.

De forma resumida, o *cat_babysitter* é um sistema de gerenciamento de reservas para serviços de cuidado de gatos, no qual é possível criar, consultar, atualizar, remover e listar agendamentos. A aplicação foi estruturada no modelo cliente-servidor: o servidor publica os métodos de manipulação de reservas e o cliente consome esses métodos remotamente, como se fossem chamadas locais.

II. DESENVOLVIMENTO

A principal diferença em relação ao primeiro trabalho é a criação do arquivo *booking.py*, responsável por definir e padronizar a estrutura dos dados trocados remotamente. Nesse arquivo, a classe *Booking* foi implementada com *@dataclass*, conforme visto abaixo:

```
from dataclasses import dataclass

@dataclass
class Booking:
    id: int | None
    owner_name: str
    cat_name: str
    days: int
    price_per_day: float
    city: str
```

Também foi feito novas funções de serialização e desserialização (*booking_to_dict* e *booking_from_dict*), permitindo converter objetos *Booking* para dicionários. Esse passo é necessário para o funcionamento do RMI com *Pyro5*, pois os objetos precisam ser transformados em um formato transmissível pela rede.

Por fim, a função *register_pyro_serializers* registra essas conversões no *Pyro5*, para que cliente e servidor interpretem corretamente os mesmos tipos de dados durante as chamadas remotas.

III. WIRESHARK

No Wireshark, utilizou-se o filtro `ip.addr == 127.0.0.1`, focando apenas no tráfego local (loopback) entre Name Server, servidor e cliente.

Para analisar o tráfego da aplicação RMI, foram adicionados pontos de pausa no código com `input(...)` para permitir capturas em momentos específicos da execução.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	127.0.0.1	127.0.0.1	TCP	60	65535 → 9090 [RST] Seq=127.0.0.1:65535 Win=0 Len=0
2	0.000129	127.0.0.1	127.0.0.1	TCP	60	9090 → 65535 [RST] Seq=127.0.0.1:9090 Win=0 Len=0
3	0.000268	127.0.0.1	127.0.0.1	TCP	60	65535 → 9090 [RST] Seq=127.0.0.1:65535 Win=0 Len=0
4	0.000394	127.0.0.1	127.0.0.1	TCP	60	9090 → 65535 [RST] Seq=127.0.0.1:9090 Win=0 Len=0
5	0.000529	127.0.0.1	127.0.0.1	TCP	60	65535 → 9090 [RST] Seq=127.0.0.1:65535 Win=0 Len=0
6	0.000659	127.0.0.1	127.0.0.1	TCP	60	9090 → 65535 [RST] Seq=127.0.0.1:9090 Win=0 Len=0
7	0.000789	127.0.0.1	127.0.0.1	TCP	60	65535 → 9090 [RST] Seq=127.0.0.1:65535 Win=0 Len=0
8	0.000919	127.0.0.1	127.0.0.1	TCP	60	9090 → 65535 [RST] Seq=127.0.0.1:9090 Win=0 Len=0
9	0.001049	127.0.0.1	127.0.0.1	TCP	60	65535 → 9090 [RST] Seq=127.0.0.1:65535 Win=0 Len=0
10	0.001179	127.0.0.1	127.0.0.1	TCP	60	9090 → 65535 [RST] Seq=127.0.0.1:9090 Win=0 Len=0

Figura 1. Captura ao rodar o nameservice e fazer associação com server:

No.	Time	Source	Destination	Protocol	Length	Info
11	0.001309	127.0.0.1	127.0.0.1	TCP	60	9090 → 65535 [RST] Seq=127.0.0.1:9090 Win=0 Len=0
12	0.001439	127.0.0.1	127.0.0.1	TCP	60	65535 → 9090 [RST] Seq=127.0.0.1:65535 Win=0 Len=0
13	0.001569	127.0.0.1	127.0.0.1	TCP	60	9090 → 65535 [RST] Seq=127.0.0.1:9090 Win=0 Len=0
14	0.001699	127.0.0.1	127.0.0.1	TCP	60	65535 → 9090 [RST] Seq=127.0.0.1:65535 Win=0 Len=0
15	0.001829	127.0.0.1	127.0.0.1	TCP	60	9090 → 65535 [RST] Seq=127.0.0.1:9090 Win=0 Len=0
16	0.001959	127.0.0.1	127.0.0.1	TCP	60	65535 → 9090 [RST] Seq=127.0.0.1:65535 Win=0 Len=0
17	0.002089	127.0.0.1	127.0.0.1	TCP	60	9090 → 65535 [RST] Seq=127.0.0.1:9090 Win=0 Len=0
18	0.002219	127.0.0.1	127.0.0.1	TCP	60	65535 → 9090 [RST] Seq=127.0.0.1:65535 Win=0 Len=0
19	0.002349	127.0.0.1	127.0.0.1	TCP	60	9090 → 65535 [RST] Seq=127.0.0.1:9090 Win=0 Len=0
20	0.002479	127.0.0.1	127.0.0.1	TCP	60	65535 → 9090 [RST] Seq=127.0.0.1:65535 Win=0 Len=0

Figura 2. Captura do handshake

No servidor, as pausas foram inseridas logo após o registro no Name Server e antes do loop principal, em *cat_server.py*. Isso permitiu observar os pacotes de associação do serviço remoto:

Vamos o *handshake* entre o cliente e o Pyro, entre os pacotes 10 e 12, conforme observado na Fig 2

No cliente, também foram adicionadas pausas após a criação do proxy, após o *bind*, conforme Fig 3, e após chamada de método remoto (*creat* com os dados conforme Fig 6). Assim foi possível separar visualmente cada etapa da comunicação cliente-servidor durante a captura;

Ao inspecionar o pacote 51, visto em Fig 4, observamos que os mesmos dados estão em 5, que são os mesmos valores

No.	Time	Source	Destination	Protocol	Length	Info
21	0.002609	127.0.0.1	127.0.0.1	TCP	60	9090 → 65535 [RST] Seq=127.0.0.1:9090 Win=0 Len=0
22	0.002739	127.0.0.1	127.0.0.1	TCP	60	65535 → 9090 [RST] Seq=127.0.0.1:65535 Win=0 Len=0
23	0.002869	127.0.0.1	127.0.0.1	TCP	60	9090 → 65535 [RST] Seq=127.0.0.1:9090 Win=0 Len=0
24	0.002999	127.0.0.1	127.0.0.1	TCP	60	65535 → 9090 [RST] Seq=127.0.0.1:65535 Win=0 Len=0
25	0.003129	127.0.0.1	127.0.0.1	TCP	60	9090 → 65535 [RST] Seq=127.0.0.1:9090 Win=0 Len=0
26	0.003259	127.0.0.1	127.0.0.1	TCP	60	65535 → 9090 [RST] Seq=127.0.0.1:65535 Win=0 Len=0
27	0.003389	127.0.0.1	127.0.0.1	TCP	60	9090 → 65535 [RST] Seq=127.0.0.1:9090 Win=0 Len=0
28	0.003519	127.0.0.1	127.0.0.1	TCP	60	65535 → 9090 [RST] Seq=127.0.0.1:65535 Win=0 Len=0
29	0.003649	127.0.0.1	127.0.0.1	TCP	60	9090 → 65535 [RST] Seq=127.0.0.1:9090 Win=0 Len=0
30	0.003779	127.0.0.1	127.0.0.1	TCP	60	65535 → 9090 [RST] Seq=127.0.0.1:65535 Win=0 Len=0

Figura 3. Captura ao realizar o proxy bind

No.	Time	Source	Destination	Protocol	Length	Info
31	0.003909	127.0.0.1	127.0.0.1	TCP	60	9090 → 65535 [RST] Seq=127.0.0.1:9090 Win=0 Len=0
32	0.004039	127.0.0.1	127.0.0.1	TCP	60	65535 → 9090 [RST] Seq=127.0.0.1:65535 Win=0 Len=0
33	0.004169	127.0.0.1	127.0.0.1	TCP	60	9090 → 65535 [RST] Seq=127.0.0.1:9090 Win=0 Len=0
34	0.004299	127.0.0.1	127.0.0.1	TCP	60	65535 → 9090 [RST] Seq=127.0.0.1:65535 Win=0 Len=0
35	0.004429	127.0.0.1	127.0.0.1	TCP	60	9090 → 65535 [RST] Seq=127.0.0.1:9090 Win=0 Len=0

Figura 4. Captura ao criar novas entradas

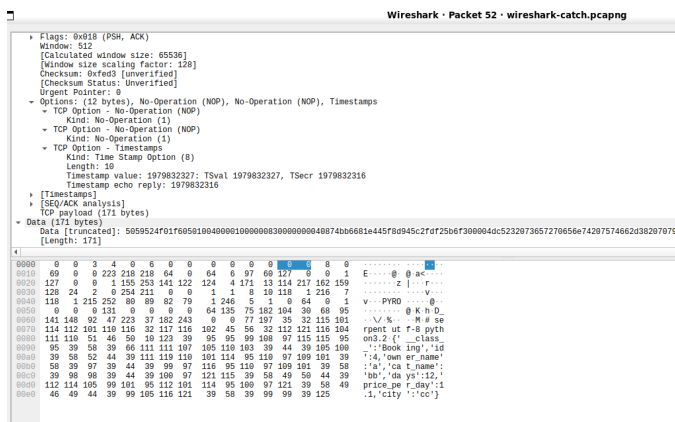


Figura 5. Captura ao receber novas entradas

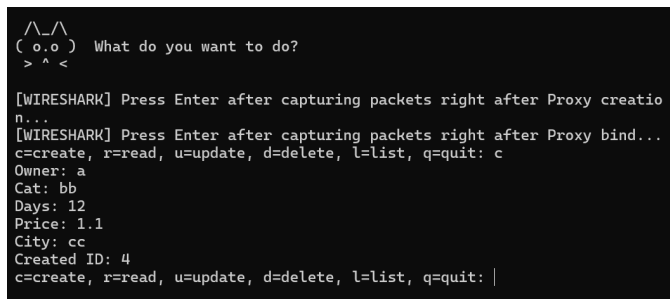


Figura 6. Dados inseridos no CRUD

inseridos pelo cliente, conforme vemos em Fig 7. Isto confirma que os dados foram transmitidos com êxito

IV. RESULTADOS E DISCUSSÃO

A análise mostrou que os dados trafegaram em formato bruto, sem criptografia na captura observada. Em especial, foi possível identificar no payload informações equivalentes as enviadas pelo cliente no CRUD, confirmando que os parâmetros de aplicação ficam visíveis no tráfego quando não há camada adicional de proteção.

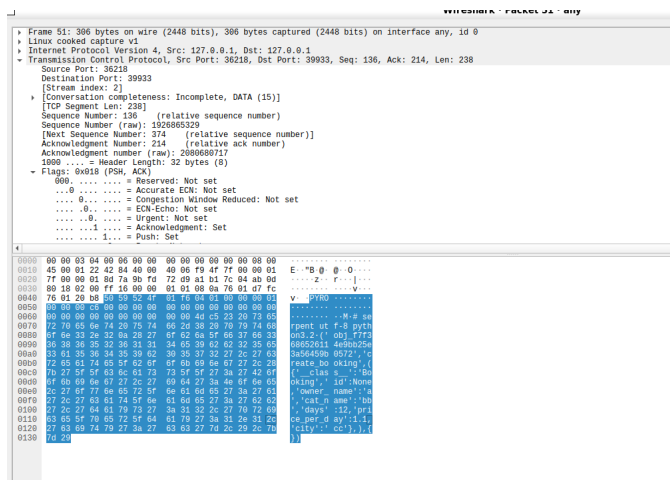


Figura 7. Dados capturados no Wireshark